Chapter 8 solutions

CH08-E1

1. T_1 linear:

$$T_1(x,y) = \begin{bmatrix} 2 & -1 \\ 1 & 3 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}.$$

Every component is a linear form.

2. Since $T_2(0,0) = (0,1) \neq (0,0)$, the map is nonlinear.

3. T_3 linear:

$$T_3(\alpha p + \beta q)(t) = t(\alpha p + \beta q)'(t) = \alpha t p'(t) + \beta t q'(t).$$

Also $tp'(t)$ still belongs to $\mathcal{P}_2$, so the map has the stated codomain.

4. Squaring is nonlinear. Set $f = g = 1$.

$$T_4(f+g) = 4 \neq 1 + 1 = T_4(f) + T_4(g).$$

CH08-E2

1.

$$A = \begin{bmatrix} 1 & 2 & -1 \\ 1 & -1 & 2 \end{bmatrix}.$$

2. Solve

$$x + 2y - z = 0, \qquad x - y + 2z = 0.$$

Subtraction gives $3y - 3z = 0$, hence $y = z$ and $x = -z$. Therefore

$$N(T) = \operatorname{span}\{(-1,1,1)^T\}, \qquad \text{nullity} = 1.$$

3. The columns $a_1 = (1,1)^T, a_2 = (2,-1)^T$ are independent, giving rank 2 and

$$C(T) = \mathbb{R}^2.$$

These two columns form an image basis.

4.

$$1 + 2 = 3 = \dim \mathbb{R}^3,$$

This verifies rank-nullity.

5. A nonzero kernel rules out injectivity. The image fills the codomain, proving surjectivity.

CH08-E3

1. Input basis:

Chapter 8 solutions (continued)

$$v_1 = (1,1)^T, \qquad v_2 = (1,-1)^T.$$
$$T(v_1) = (3,1,1)^T, \qquad T(v_2) = (-1,-1,1)^T.$$

Standard output coordinates:

$$M_1 = \begin{bmatrix} 3 & -1 \\ 1 & -1 \\ 1 & 1 \end{bmatrix}.$$

2. New output basis:

$$B_{\text{out}} = \begin{bmatrix} 1 & 1 & 1 \\ 0 & 1 & 1 \\ 0 & 0 & 1 \end{bmatrix}, \qquad B_{\text{out}}^{-1} = \begin{bmatrix} 1 & -1 & 0 \\ 0 & 1 & -1 \\ 0 & 0 & 1 \end{bmatrix}.$$
$$M_2 = B_{\text{out}}^{-1} M_1 = \begin{bmatrix} 2 & 0 \\ 0 & -2 \\ 1 & 1 \end{bmatrix}.$$

3. Formula:

$$M_2 = B_{\text{out}}^{-1} A B_{\text{in}}.$$

The first column means

$$T(v_1) = 2w_1 + w_3.$$

The second means

$$T(v_2) = -2w_2 + w_3.$$

Lin Cing https://github.com/xinhi6716/computer-science-self-study

CH08-E4

1. Trace 5 and determinant 6 give eigenvalues 3 and 2 in this two-dimensional case.

$$\lambda = 3: \quad (A - 3I)x = \begin{bmatrix} 1 & -2 \\ 1 & -2 \end{bmatrix} x = 0,$$

Choose $v_1 = (2,1)^T$.

$$\lambda = 2: \quad (A - 2I)x = \begin{bmatrix} 2 & -2 \\ 1 & -1 \end{bmatrix} x = 0,$$

Choose $v_2 = (1,1)^T$.

2.

$$B = \begin{bmatrix} 2 & 1 \\ 1 & 1 \end{bmatrix}, \qquad B^{-1} = \begin{bmatrix} 1 & -1 \\ -1 & 2 \end{bmatrix}.$$

3.

$$B^{-1} A B = \begin{bmatrix} 3 & 0 \\ 0 & 2 \end{bmatrix}.$$

Chapter 8 solutions (continued)

Trace 5 and determinant 6 agree with the original matrix.

4. Scaling each v_i by $s_i \neq 0$ gives

$$B' = B\,\mathrm{diag}(s_1, s_2).$$

The diagonal scaling commutes with the eigenvalue matrix and cancels with its inverse, leaving

$$B'^{-1} A B' = \mathrm{diag}(3, 2).$$

CH08-E5

1. Input $(1, t, t^2)$, output $(1, t)$:

$$M_1 = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 2 \end{bmatrix}.$$

2. The kernel is the constant polynomials and the image is $\mathcal{P}_1$.

$$N(D) = \mathrm{span}\{1\}, \qquad C(D) = \mathrm{span}\{1, t\}.$$

Thus $3 = 1 + 2$, as rank-nullity requires.

3. Set $s = t - 1$.

$$D(1) = 0, \qquad D(s) = 1, \qquad D(s^2) = 2s = -2 + 2t.$$

Hence

$$M_2 = \begin{bmatrix} 0 & 1 & -2 \\ 0 & 0 & 2 \end{bmatrix}.$$

4. The matrices are 2×3, so similarity is not defined for them. They represent the same map in different domain bases, related by a two-basis conversion.

CH08-E6

1. The triangular matrix has eigenvalue 0 with algebraic multiplicity 2.

$$Nx = 0$$

The equation imposes $x_2 = 0$, so the eigenspace is

$$\mathrm{span}\{(1, 0)^T\}.$$

2. Generalized vector:

$$w = (0, 1)^T, \qquad Nw = (1, 0)^T.$$

3. The given matrix is already a Jordan block.

$$N = J_2(0),$$

Take $B = I$.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Chapter 8 solutions (continued)

4. Geometric multiplicity 1 is below algebraic multiplicity 2, preventing an eigenbasis. Equal eigenvalues do not imply a scalar multiple of the identity.

CH08-E7

1. With the specified left cyclic coordinate shift P:

$$P = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \end{bmatrix}.$$

For each $v_\lambda = (1, \lambda, \lambda^2, \lambda^3)^T$,

$$Pv_\lambda = (\lambda, \lambda^2, \lambda^3, 1)^T = \lambda v_\lambda$$

because $\lambda^4 = 1$.

2. Suppose

$$\sum c_\lambda v_\lambda = 0,$$

The four coordinate equations form a Vandermonde system at distinct roots, with nonzero determinant. Equivalently, eigenvectors for distinct eigenvalues are independent. Thus all coefficients vanish.

3.

$$Cv_\lambda = (c_0 + c_1\lambda + c_2\lambda^2 + c_3\lambda^3)v_\lambda.$$

These polynomial evaluations at the four roots are the diagonal entries in the Fourier basis.

4. The complex eigenvalues occur in conjugate pairs. Here the spectrum is $1, i, -1, -i$. Pairing conjugate eigenvectors yields a real sine-cosine representation.

CH08-E8

1.

$$B_\epsilon = \begin{bmatrix} 1 & 1 \\ 0 & \epsilon \end{bmatrix}, \qquad \det B_\epsilon = \epsilon.$$

The determinant is nonzero for $\epsilon > 0$, proving a basis.

2. Columns

$$b_1 = (1, 0)^T, \qquad b_2 = (1, \epsilon)^T.$$

$$B_\epsilon^T B_\epsilon = \begin{bmatrix} 1 & 1 \\ 1 & 1 + \epsilon^2 \end{bmatrix}.$$

Chapter 8 solutions (continued)

Its eigenvalues are

$$\lambda_{\pm} = \frac{2 + \epsilon^2 \pm \sqrt{4 + \epsilon^4}}{2}.$$

Therefore

$$\kappa_2(B_\epsilon) = \sqrt{\frac{2 + \epsilon^2 + \sqrt{4 + \epsilon^4}}{2 + \epsilon^2 - \sqrt{4 + \epsilon^4}}}.$$

As $\epsilon \to 0^+$, $\lambda_- \to 0$ while $\lambda_+ \to 2$, so the condition number diverges. For numerical evaluation, use $\lambda_- = \epsilon^2/\lambda_+$ to avoid subtractive cancellation.

3. Angle:

$$\cos\theta = \frac{b_1^T b_2}{\|b_1\|\|b_2\|} = \frac{1}{\sqrt{1 + \epsilon^2}}.$$

The angle tends to zero as $\epsilon \to 0^+$.

4. A coordinate perturbation δc gives

$$\|B_\epsilon \delta c\| \le \|B_\epsilon\|_2 \|\delta c\|.$$

If $\|\delta c\|_2 \le 10^{-8}$, reconstruction error is at most $\sqrt{\lambda_+}\, 10^{-8}$. If each coordinate error is bounded by 10^{-8}, the bound is $\sqrt{2\lambda_+}\, 10^{-8}$. In the reverse conversion, errors can be amplified by $\|B_\epsilon^{-1}\|_2 = 1/\sqrt{\lambda_-}$, which diverges as $\epsilon \to 0^+$. Exact invertibility does not guarantee numerically reliable coordinates.

CH08-E9

1.

$$A = \begin{bmatrix} 1 & 1 \\ 1 & -1 \\ 2 & 0 \end{bmatrix}, \qquad N = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & -2 \end{bmatrix}.$$

2. Since $(S \circ T)(v) = S(T(v))$, apply A first, then N.

$$NA = \begin{bmatrix} 3 & 1 \\ -3 & -1 \end{bmatrix}.$$

3. $T(1, 2) = (3, -1, 2)$, so $S(3, -1, 2) = (5, -5)^T$.

$$(S \circ T)(1, 2) = \begin{bmatrix} 3 & 1 \\ -3 & -1 \end{bmatrix} \begin{bmatrix} 1 \\ 2 \end{bmatrix} = \begin{bmatrix} 5 \\ -5 \end{bmatrix}.$$

Both computations agree.

Chapter 8 solutions (continued)

Verification code appendix: basis_change.py

```
"""Check change-of-basis matrices and a differentiation example."""

from fractions import Fraction

Mat = tuple[tuple[float, ...], ...]
Vec = tuple[float, ...]


def transpose(a: Mat) -> Mat:
    return tuple(zip(*a))


def matmul(a: Mat, b: Mat) -> Mat:
    return tuple(
        tuple(
            sum(a[i][k] * b[k][j] for k in range(len(b)))
            for j in range(len(b[0]))
        )
        for i in range(len(a))
    )


def matvec(a: Mat, x: Vec) -> Vec:
    return tuple(sum(a[i][j] * x[j] for j in range(len(x))) for i in range(len(a)))


def inverse2(a: Mat) -> Mat:
    """Use exact rational arithmetic on stored finite values, then convert to float; not a
general numerical solver."""
    p, q, r, s = (Fraction(a[i][j]) for i, j in ((0, 0), (0, 1), (1, 0), (1, 1)))
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Chapter 8 solutions (continued)

```
    determinant = p * s - q * r
    if determinant == 0:
        raise ValueError("exactly singular matrix for the supplied values")
    return (
        (float(s / determinant), float(-q / determinant)),
        (float(-r / determinant), float(p / determinant)),
    )


scaled_identity = ((1e-8, 0.0), (0.0, 1e-8))
assert inverse2(scaled_identity) == ((1e8, 0.0), (0.0, 1e8))
try:
    inverse2(((1.0, 2.0), (2.0, 4.0)))
except ValueError:
    pass
else:
    raise AssertionError("a truly singular matrix must be rejected")


def clean(a: Mat) -> Mat:
    return tuple(
        tuple(0.0 if abs(value) < 1e-12 else round(value, 12) for value in row)
        for row in a
    )


a = ((3.0, 1.0), (1.0, 3.0))
b = ((1.0, 1.0), (1.0, -1.0))
b_inv = inverse2(b)
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Chapter 8 solutions (continued)

```
diagonal = matmul(matmul(b_inv, a), b)
assert clean(diagonal) == ((4.0, 0.0), (0.0, 2.0))

standard_x = (2.0, 0.0)
basis_x = matvec(b_inv, standard_x)
assert basis_x == (1.0, 1.0)
power_on_basis = (4**5, 2**5)
power_x = matvec(b, power_on_basis)
assert power_x == (1056.0, 992.0)

derivative = ((0.0, 1.0, 0.0), (0.0, 0.0, 2.0))
p_basis = (2.0, 3.0, -1.0)
derivative_coordinates = matvec(derivative, p_basis)
assert derivative_coordinates == (3.0, -2.0)

cyclic = (
    (0.0, 1.0, 0.0, 0.0),
    (0.0, 0.0, 1.0, 0.0),
    (0.0, 0.0, 0.0, 1.0),
    (1.0, 0.0, 0.0, 0.0),
)
fourier_vector = (1.0 + 0.0j, 1j, -1.0 + 0.0j, -1j)
image = tuple(sum(cyclic[i][j] * fourier_vector[j] for j in range(4)) for i in range(4))
assert image == tuple(1j * value for value in fourier_vector)

for epsilon in (0.1, 0.5, 1.0, 2.0):
    trace = 2.0 + epsilon**2
    discriminant = (4.0 + epsilon**4) ** 0.5
    large = (trace + discriminant) / 2.0
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Chapter 8 solutions (continued)

```
    small = epsilon**2 / large
    assert abs(large + small - trace) < 1e-12
    assert abs(large * small - epsilon**2) < 1e-12
    gram = ((1.0, 1.0), (1.0, 1.0 + epsilon**2))
    eigenvector = (1.0, large - 1.0)
    assert all(abs(matvec(gram, eigenvector)[i] - large * eigenvector[i]) < 1e-12 for i in
range(2))

print("eigenbasis matrix =", diagonal)
print("T^5(2,0) =", power_x)
print("D(2+3t-t^2) coordinates =", derivative_coordinates)
print("Fourier eigenpair checked")

base_matrix = ((2.0, 1.0), (1.0, 2.0))
basis = ((1.0, 1.0), (1.0, -1.0))
changed = matmul(matmul(b_inv, base_matrix), basis)
assert clean(changed) == ((3.0, 0.0), (0.0, 1.0))
coordinate_x = matvec(b_inv, (2.0, 0.0))
assert coordinate_x == (1.0, 1.0)
assert matvec(basis, matvec(changed, coordinate_x)) == matvec(base_matrix, (2.0, 0.0))

t_matrix = ((1.0, 1.0), (1.0, -1.0), (2.0, 0.0))
s_matrix = ((1.0, 0.0, 1.0), (0.0, 1.0, -2.0))
composed = matmul(s_matrix, t_matrix)
assert clean(composed) == ((3.0, 1.0), (-3.0, -1.0))
assert matvec(composed, (1.0, 2.0)) == matvec(s_matrix, matvec(t_matrix, (1.0, 2.0)))
print("basis change and composition checks passed")
```

References and attribution

Gilbert Strang. Introduction to Linear Algebra, 5th Edition, 2016.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study